

ORBITAL TRAJECTORY SIMULATION USING FOURTH-ORDER RUNGE-KUTTA (RK4) METHOD

By: Joana Lawer

Abstract

Accurate prediction of satellite motion is a foundational problem in astrodynamics and space mission analysis. This project presents the development of a numerical orbital trajectory simulation based on Newton's Law of Universal Gravitation and integrated using the Fourth-Order Runge-Kutta (RK4) method. The Earth-satellite system is formulated as a two-body problem governed by Newton's Law of Universal Gravitation. The resulting second-order equations of motion are transformed into a first-order state-space system and integrated numerically using RK4 method to advance the system in time.

1. Introduction

Orbital mechanics lies at the core of modern space systems, enabling applications ranging from satellite mission design, control, communication to Earth observation and navigation. As space activities expand globally, including Ghana seeking to make strides in this field, the need for more local expertise in orbital modeling and simulation becomes very important. Ghana's first satellite, GhanaSat-1, demonstrates the growing relevance of orbital modelling capabilities in the region.

2. Mathematical Formulation

2.1 Newton's Law of Gravitation

The motion of objects in space is governed by Newton's Law of Universal Gravitation, which describes the force of attraction between two masses.

$$F = G \frac{m_1 m_2}{r^2}$$

In this project the Earth-satellite system is modeled as a two-body problem giving the force of attraction between the Earth and the orbiting satellite as:

$$F = G \frac{Mm}{r^2}$$

where:

- G is the universal Gravitational Constant
- M is the Mass of the Earth
- m is the mass of the satellite
- r is the magnitude (distance) between the two bodies.

2.2 Gravitational Force to Gravitational Acceleration

Newton's second law can be summed up in a simple mathematical equation where Force (F) is equivalent to the mass of an object (m) multiplied by its acceleration (a);

$$F = ma$$

Applying this on the satellite it gives:

- m is the mass of the satellite
- a is the acceleration of the satellite

Substituting both equations: $F = G \frac{Mm}{r^2}$ and $F = ma$, the acceleration of the satellite becomes:

$$a = \frac{GM}{r^2},$$

representing the scalar magnitude of gravitational acceleration. The mass of the satellite cancels out in both equations which indicates that gravitational acceleration is independent of satellite mass.

2.3 Vector Representation

Gravitational force is always attractive and pulling objects in orbit toward the centre, in this case towards the Earth's. The magnitude of the satellite gravitational acceleration follows an inverse-square law which is mathematically represented with a negative sign to show that acceleration acts in the opposite direction of the position vector, which points away from the centre.

The unit vector is used to define the direction of the position vector: $\hat{r} = \mathbf{r}/\|\mathbf{r}\|$.

The scalar magnitude is multiplied by the negative unit vector: $a = \frac{GM}{r^2} * -\frac{r}{\|\mathbf{r}\|}$

Acceleration vector then becomes:

$$a = -\frac{GM}{\|\mathbf{r}\|^3} \mathbf{r}$$

Gravitational parameter is defined as $\mu = GM$. Hence the equation for acceleration of the satellite can be written as:

$$\mathbf{a} = -\frac{\mu}{\|\mathbf{r}\|^3} \mathbf{r}$$

2.4 State-Space Representation

Acceleration (a), is basically the change in velocity (v), with respect to time (t), representing first derivative of velocity. Velocity is a first derivative of position (x) with respect to time therefore making acceleration the second derivative of position.

Orbital motion is governed by second-order ordinary differential equations.

$$a(t) = \frac{dv(t)}{dt}$$

$$v(t) = \frac{dx(t)}{dt}$$

$$a(t) = \frac{dv(t)}{dt} = \frac{d^2x(t)}{dt^2} = -\frac{\mu}{r^3} \vec{r}$$

For numerical integration, the state variables position and velocity are used to turn second-order ordinary differential equations into coupled first-order ordinary differential equations. For a three-dimensional (3D) orbit, x, y, z are the Cartesian coordinates of the satellite's position in three-dimensional space. The variables are:

$$\mathbf{r} = (x, y, z) \text{ and } \mathbf{v} = (v_x, v_y, v_z)$$

The distance, r , is calculated as components of magnitude: $\|r\| = \sqrt{x^2 + y^2 + z^2}$

Hence the 3D state-space representation of the system function derivative is:

$$\frac{dy}{dt} = f(t, y) = \begin{bmatrix} x \\ y \\ z \\ v_x \\ v_y \\ v_z \end{bmatrix} = \begin{bmatrix} v_x \\ v_y \\ v_z \\ a_x \\ a_y \\ a_z \end{bmatrix} = \begin{bmatrix} v_x \\ v_y \\ v_z \\ -\frac{\mu * x}{(x^2 + y^2 + z^2)^{\frac{3}{2}}} \\ -\frac{\mu * y}{(x^2 + y^2 + z^2)^{\frac{3}{2}}} \\ -\frac{\mu * z}{(x^2 + y^2 + z^2)^{\frac{3}{2}}} \end{bmatrix}$$

3. Numerical Methodology

The Runge-Kutta Fourth Order Method (RK4)

Because gravitational acceleration depends on position and position evolves continuously over time, analytical solutions are insufficient for general cases. Numerical integration is then applied to enable step-by-step propagation of the state of the satellite.

The Fourth-Order Runge-Kutta (RK4) method is used to numerically integrate the coupled state equations. RK4 was selected due to its higher accuracy and stability compared to lower-order methods, particularly for conservative dynamical systems such as orbital motion.

The entire system of a satellite orbiting the Earth is bundled into a single state-vector, $y = [x, y, z, v_x, v_y, v_z]^T$ and the first-order differential equation, $\frac{dy}{dt} = f(t, y)$ as shown above.

Applying the Fourth-Order Runge-Kutta Method

The RK4 method advances the state from time t_n to $t_{n+1} = t_n + \Delta t$ by computing four intermediate slope estimates for both position and velocity. Where n is the current step and $n+1$ is the next step.

Given the state-vector and system of the derivative function, the update from t_n to t_{n+1} is calculated as:

$$k_1 = f(t_n, y_n) \dots \dots \dots \text{slope (1)}$$

$$k_2 = f\left(t_n + \frac{\Delta t}{2}, y_n + \frac{\Delta t}{2} k_1\right) \dots \dots \dots \text{slope (2)}$$

$$k_3 = f\left(t_n + \frac{\Delta t}{2}, y_n + \frac{\Delta t}{2} k_2\right) \dots \dots \dots \text{slope (3)}$$

$$k_4 = f(t_n + \Delta t, y_n + \Delta t k_3) \dots \dots \dots \text{slope (4)}$$

The weighted average computed to get the final state is given as:

$$y_{n+1} = y_n + \frac{\Delta t}{6} (k_1 + 2k_2 + 2k_3 + k_4)$$

where:

- t_n : the time at the beginning of the current step.
- y_n : the satellite's state (position/velocity) at time
- t_{n+1} : the time at the end of the current step ().
- y_{n+1} : the new, updated state of the satellite that is calculated using the RK4 method.

Here RK4 method is being used to answer the question:

“If I know the position and velocity now, where will the satellite be after a short time, Δt ?”

The slopes are calculated as:

First slope: Where is the satellite now?

$$k_1 = \begin{bmatrix} k_1, x \\ k_1, y \\ k_1, z \\ k_1, v_x \\ k_1, v_y \\ k_1, v_z \end{bmatrix} = \begin{bmatrix} v_x \\ v_y \\ v_z \\ a_x(x, y, z) \\ a_y(x, y, z) \\ a_z(x, y, z) \end{bmatrix}$$

Second slope: Where is it after a half step forward?

$$k_2 = \begin{bmatrix} k_2, x \\ k_2, y \\ k_2, z \\ k_2, v_x \\ k_2, v_y \\ k_2, v_z \end{bmatrix} = \begin{bmatrix} v_x + \frac{\Delta t}{2} k_1, v_x \\ v_y + \frac{\Delta t}{2} k_1, v_y \\ v_z + \frac{\Delta t}{2} k_1, v_z \\ a_x(x + \frac{\Delta t}{2} k_1, x, y, z) \\ a_y(x, y + \frac{\Delta t}{2} k_1, y, z) \\ a_z(x, y, z + \frac{\Delta t}{2} k_1, z) \end{bmatrix}$$

Third slope: Where is after another half step forward?

$$k_3 = \begin{bmatrix} k_3, x \\ k_3, y \\ k_3, z \\ k_3, v_x \\ k_3, v_y \\ k_3, v_z \end{bmatrix} = \begin{bmatrix} v_x + \frac{\Delta t}{2} k_2, v_x \\ v_y + \frac{\Delta t}{2} k_2, v_y \\ v_z + \frac{\Delta t}{2} k_2, v_z \\ a_x(x + \frac{\Delta t}{2} k_2, x, y, z) \\ a_y(x, y + \frac{\Delta t}{2} k_2, y, z) \\ a_z(x, y, z + \frac{\Delta t}{2} k_2, z) \end{bmatrix}$$

Fourth slope: What happens at the end of the step?

$$k_4 = \begin{bmatrix} k_4, x \\ k_4, y \\ k_4, z \\ k_4, v_x \\ k_4, v_y \\ k_4, v_z \end{bmatrix} = \begin{bmatrix} v_x + \Delta t * k_3, v_x \\ v_y + \Delta t * k_3, v_y \\ v_z + \Delta t * k_3, v_z \\ a_x(x + \Delta t * k_3, v_x, y, z) \\ a_y(x, y + \Delta t * k_3, v_y, z) \\ a_z(x, y, z + \Delta t * k_3, v_z) \end{bmatrix}$$

Weighted average: To give accurate position and velocity update.

$$x_{n+1} = x_n + \frac{\Delta t}{6} (k_{1,x} + 2k_{2,x} + 2k_{3,x} + k_{4,x})$$

$$y_{n+1} = y_n + \frac{\Delta t}{6} (k_{1,y} + 2k_{2,y} + 2k_{3,y} + k_{4,y})$$

$$z_{n+1} = z_n + \frac{\Delta t}{6} (k_{1,z} + 2k_{2,z} + 2k_{3,z} + k_{4,z})$$

$$v_{x,n+1} = v_{x,n} + \frac{\Delta t}{6} (k_{1,v_x} + 2k_{2,v_x} + 2k_{3,v_x} + k_{4,v_x})$$

$$v_{y,n+1} = v_{y,n} + \frac{\Delta t}{6} (k_{1,v_y} + 2k_{2,v_y} + 2k_{3,v_y} + k_{4,v_y})$$

$$v_{z,n+1} = v_{z,n} + \frac{\Delta t}{6} (k_{1,v_z} + 2k_{2,v_z} + 2k_{3,v_z} + k_{4,v_z})$$

Finally, weighted average of these slopes yields the updated state of the satellite, that is the position and velocity. Acceleration is recomputed at each intermediate stage since it depends on position. This allows RK4 to accurately capture orbital curvature within a single time step while significantly reducing truncation error.

4. Results and Validation

4.1 Simulation of Orbital Trajectory in 3D

The orbital trajectory of a satellite around the Earth is demonstrated using python. A three-dimensional orbital propagation was performed using the following parameters depicting the International Space Station orbit:

- Earth's gravitational parameter μ given as: $(3.986 * 10^{14} m^3/s^2)$
- Earth's radius R_E : $(6.371 * 10^6 m)$
- Initial altitude set to: $(400km = 400000m)$
- Initial position is taken to be $(r_0 = R_E + 400000)$
- Initial velocity chosen to be $v_0 = (7670)m/s$
- Time step is set to an interval of 10 seconds: $\Delta t = 10s$
- Total propagation time (Approximate time it takes to make an orbit): **5600s**

In this simulation, atmospheric drag and the Earth's own rotation are neglected.

[Link](#) to code.

4.2 Analytical Comparison

The velocity of a circular orbit is given by: $v_{circ} = \sqrt{\frac{\mu}{r}}$, where r is the position.

Using the parameters stated earlier:

$$\begin{aligned}r &= R_E + altitude \\r &= 6.371 * 10^6 m + 400000 m \\r &= 6371000 m + 400000 m \\r &= 6771000 m \\r &= 6.771 * 10^6 m\end{aligned}$$

Substituting this to get the velocity, it yields:

$$\begin{aligned}v_{circ} &= \sqrt{\frac{3.986 * 10^{14} m^3/s^2}{6.771 * 10^6 m}} \\v_{circ} &= \sqrt{\frac{3.986}{6.771} * 10^8 m^2/s^2} \\v_{circ} &= \sqrt{0.5886870477 * 10^8 m^2/s^2} \\v_{circ} &= \sqrt{58868704.77} m/s \\v_{circ} &= 7672.5944 m/s \\v_{circ} &= 7673 m/s\end{aligned}$$

The resulting velocity in a nearly circular orbit is **7673m/s** which closely matches the chosen initial velocity.

Comparing the orbital period, time = T , for the circular orbit using: $T = 2\pi \sqrt{\frac{r^3}{\mu}}$

$$\begin{aligned}T &= 2\pi \sqrt{\frac{(6.771 * 10^6 m)^3}{3.986 * 10^{14} m^3/s^2}} \\T &= 2\pi \sqrt{\frac{310.426252 * 10^{18} m^3}{3.986 * 10^{14} m^3/s^2}} \\T &= 2\pi \sqrt{\frac{310.426}{3.986} * 10^4 s^2} \\T &= 2\pi \sqrt{77.8791 * 10^4 s^2} \\T &= 2\pi * 882.4913 s \\T &= 5544.8567 s\end{aligned}$$

The simulated trajectory completes one orbit in approximately duration of **5545** seconds, confirming physical consistency of the numerical integration. Runge-Kutta fourth order method has local truncation error $\mathcal{O}(\Delta t^5)$ and is theoretically fourth-order accurate, meaning its global error scales as $\mathcal{O}(\Delta t^4)$. The small discrepancy between analytical and simulated orbital period confirms that the timestep chosen is sufficiently small for high-accuracy propagation.

4.3 Numerical Accuracy and Stability Considerations

Energy Conservation

In an ideal two-body gravitational problem, the total mechanical energy of the system should remain constant over time. The total energy is given by the sum of kinetic and gravitational potential energy:

$$E = \frac{1}{2}mv^2 - \frac{\mu m}{r}$$

where v is the magnitude of velocity and r is the distance from the Earth's centre. In numerical simulations, small deviations from exact energy conservation may occur due to truncation errors introduced by the integration method. However, when the timestep is sufficiently small, the RK4 method maintains good short-term stability and produces trajectories that remain close to the true physical orbit.

Timestep Sensitivity

The accuracy of numerical integration strongly depends on the size of the timestep Δt . A smaller timestep generally produces more accurate results because the numerical approximation more closely follows the true continuous trajectory of the system. However, smaller timesteps also increase computational cost since more iterations are required. In this simulation a timestep of $\Delta t = 10\text{s}$, provides a good balance between accuracy and computational efficiency for a low Earth orbit. If a much larger timestep were used, the numerical solution could gradually deviate from the true orbit, leading to distortions such as orbital drift or instability. On the other hand, decreasing the timestep further would improve precision at the expense of longer computation time.

Limitations of the RK4 Method

Although the Fourth-Order Runge–Kutta method provides high accuracy for many ordinary differential equation problems, does not inherently preserve the geometric structure of systems such as orbital mechanics over very long simulation times. As a result, small numerical errors can accumulate and lead to gradual energy drift in long-term integrations. For extended orbital simulations, specialized methods such as symplectic integrators (e.g., the Verlet or Leapfrog methods) are often preferred because they better preserve the physical invariants of the system. In spite of that, RK4 remains a reliable and widely used method for short to moderate propagation intervals, due to its simplicity and strong accuracy properties.

5. Conclusion

The numerical results demonstrate that the RK4 method accurately reproduces the dynamics of the two-body gravitational problem over short to moderate time intervals. This project demonstrates how mathematical modeling, differential equations, and numerical analysis combine to solve real-world physical systems. It also highlights the importance of understanding both accuracy and structural properties of numerical integrators when simulating conservative dynamical systems.

6. References

- Cheney, W., & Kincaid, D. (2008). *Numerical Mathematics and Computing*. Brooks/Cole.
- Johansson, R. (2019). *Numerical Python: Scientific Computing and Data Science Applications with NumPy, SciPy and Matplotlib*. Apress.
- Newton's Law of Motion. *NASA Glenn Research Center*.
- Orbital Mechanics. *Planetary Parameters*.
- Runge–Kutta Methods. *Wikipedia*.
- GhanaSat-1. *Wikipedia*.
- Satellites. *Wikipedia*.